

Clase 1 · Instalar las herramientas y entender qué hay detrás

9 de octubre de 2026, 12:00 a. m.

Objetivos de la sesión

1. Instalar y comprobar VS Code, Docker y jmd en tu máquina
2. Explicar qué es un agente de IA y qué son un proveedor, un modelo y sus parámetros
3. Conocer RTK, caveman, MCP y las skills, y escribir tu primera skill

En el LMS: marca cada paso del ciclo de aprendizaje, abre los enlaces, pregunta en el hilo de la sesión y entrega la tarea.

Código de la sesión: <https://github.com/davrv93/jmd> (main) · en la terminal: `jmd lesson open pa-01`

Ciclo de aprendizaje

Cada paso tiene su herramienta y una comprobación. Hazlos en orden y márcalos en el LMS.

1. Ver la clase Zoom

Asiste en vivo o mira la grabación. Ten a mano la terminal de VS Code para ir haciendo lo mismo.

Comprobación: `Has visto la clase completa (o la grabación)`

2. Instalar VS Code VS Code

Instala Visual Studio Code y abre una terminal dentro (Ctrl+ñ / Ctrl+`). En Windows, instala también la extensión WSL si vas a trabajar en Ubuntu.

Comprobación: `code --version`

3. Instalar Docker Docker

Docker Desktop (Windows y macOS) o Docker Engine (Linux). Reinicia la sesión y comprueba que el demonio responde.

Comprobación: `docker run --rm hello-world`

4. Instalar jmd y conectarlo al gateway jmd

Ejecuta el instalador de tu sistema, cierra y abre la terminal, y haz `jmd login` con la URL y la clave que te dio el instructor.

Comprobación: `jmd status`

5. Conectar tu agente Claude Code

Instala Claude Code (`npm i -g @anthropic-ai/claude-code`) u OpenCode y conéctalo al gateway con `jmd setup claude` o `jmd setup opencode`.

Comprobación: `jmd status` (Claude Code u OpenCode en verde)

6. Practicar con el agente Terminal

En una carpeta vacía, pide al agente «crea un script que imprima la fecha y la hora en español» y mira qué archivos crea y qué comandos ejecuta antes de aceptar.

Comprobación: `El script existe y corre`

7. Escribir tu primera skill Claude Code

Crea ``.claude/skills/saludo/SKILL.md`` con name y description en el front matter y una instrucción corta. Pídele al agente que la use.

Comprobación: `El agente responde usando tu skill`

8. Preguntar o responder LMS

Publica al menos una pregunta sobre la clase en este LMS (abajo) o responde una de un compañero.

Comprobación: `Tu pregunta aparece en el hilo de la sesión`

9. Entregar la tarea 1 git

Sube tu evidencia a un repositorio y entrégalo desde el LMS o con `jmd submit tarea-01`.

Comprobación: `La entrega aparece en «Mis notas» con estado en cola`

Enlaces y materiales

Instalar y comprobar VS Code, Docker y jmd en tu máquina

- [Guía de instalación de jmd \(Windows, WSL y macOS\)](#) documentación · <https://github.com/davrv93/jmd/blob/main/docs/INSTALACION.md>
- [Releases de jmd \(binarios\)](#) enlace · <https://github.com/davrv93/jmd/releases>
- [Descargar Visual Studio Code](#) enlace · <https://code.visualstudio.com/download>
- [Descargar Docker Desktop](#) enlace · <https://www.docker.com/products/docker-desktop/>
- [Docker Engine en Linux \(sin Desktop\)](#) documentación · <https://docs.docker.com/engine/install/>
- [WSL en Windows \(instalar Ubuntu\)](#) documentación · <https://learn.microsoft.com/windows/wsl/install>

Explicar qué es un agente de IA y qué son un proveedor, un modelo y sus parámetros

- [Claude Code · documentación](#) documentación · <https://docs.claude.com/en/docs/claude-code/overview>
- [OpenCode](#) enlace · <https://opencode.ai>
- [OpenRouter · modelos y capa gratuita](#) enlace · <https://openrouter.ai/models>
- [Google AI Studio \(clave de Gemini\)](#) enlace · <https://aistudio.google.com/>

Conocer RTK, caveman, MCP y las skills, y escribir tu primera skill

- [Claude Code · documentación](#) documentación · <https://docs.claude.com/en/docs/claude-code/overview>
- [RTK \(Rust Token Killer\)](#) repositorio · <https://github.com/rtk-ai/rtk>
- [caveman \(plugin de Claude Code\)](#) repositorio · <https://github.com/JuliusBrussee/caveman>
- [Model Context Protocol \(MCP\) · especificación e introducción](#) documentación · <https://modelcontextprotocol.io/>
- [Agent Skills · especificación](#) documentación · <https://agentskills.io/>
- [Skills de ejemplo de Anthropic \(repositorio\)](#) repositorio · <https://github.com/anthropics/skills>

Contenido de la clase

Avance de la clase

Hoy dejamos el entorno listo y ponemos nombre a las piezas con las que vamos a trabajar todo el curso. Al terminar deberías poder abrir VS Code, tener Docker corriendo, tener `jmd` conectado al gateway de la clase y un agente (Claude Code u OpenCode) hablando a través de él. Y deberías poder explicar, con tus palabras, qué es un agente, un proveedor, un modelo, un parámetro, qué hacen RTK y caveman, qué es MCP y qué es una skill.

Orden de la sesión:

1. Instalar VS Code · 2. Instalar Docker · 3. Instalar `jmd` y conectarlo · 4. Conceptos: agente, proveedor, modelo, parámetros · 5. Ahorrar tokens: RTK y caveman · 6. MCP · 7. Skills y tu primera skill · 8. Tarea 1.

Si algo no te funciona en clase, no te quedes atrás: sigue mirando y publica la pregunta en el hilo de abajo con el mensaje de error completo. Lo resolvemos al final o en la siguiente.

1. VS Code

Qué es VS Code

Visual Studio Code es un editor de código gratuito y de código abierto, de Microsoft. No es un «IDE» pesado: arranca rápido, y lo que le falta se instala como **extensiones** (lenguajes, Docker, WSL, Git, temas). Para nosotros importan tres cosas:

- **La terminal integrada.** Casi todo lo que haremos (instalar, `jmd`, lanzar el agente) pasa en una terminal. Abrirla dentro de VS Code evita cambiar de ventana: `Ctrl+ñ` en teclado español, `Ctrl+`` en teclado inglés, o *Terminal* → *New Terminal*.
- **La paleta de comandos** (`Ctrl+Shift+P` / `Cmd+Shift+P`): escribe lo que quieras hacer.
- **El explorador y el control de versiones** (Git) integrados: verás en vivo los archivos que el agente crea o cambia, con su diff, antes de aceptarlos. Esa revisión es parte del oficio.

Instalar VS Code

Sistema	Cómo
Windows	Descarga el instalador (<i>User Installer</i> , 64-bit) desde code.visualstudio.com y acepta «Agregar al PATH». Si trabajarás en WSL, instala la extensión WSL y abre la carpeta con «WSL: Ubuntu» abajo a la izquierda
macOS	Descarga el <code>.zip</code> , arrastra <i>Visual Studio Code</i> a <i>Aplicaciones</i> . Para tener <code>code</code> en la terminal: paleta → <i>Shell Command: Install 'code' command in PATH</i>
Linux	<code>.deb</code> / <code>.rpm</code> desde la web, o <code>snap install code --classic</code>

Comprobación:

```
code --version
```

Extensiones recomendadas para el curso: **WSL** (solo Windows), **Docker**, **GitLens** (opcional) y la del agente que uses (**Claude Code** tiene extensión; OpenCode se usa desde la terminal).

2. Docker

Qué es Docker

Docker empaqueta una aplicación con todo lo que necesita (sistema base, librerías, binarios, configuración) en una **imagen**, y la ejecuta como un **contenedor**: un proceso aislado que cree estar en su propia máquina pero comparte el núcleo con tu sistema. Es más ligero que una máquina virtual y, sobre todo, **reproducible**: la misma imagen corre igual en tu portátil, en el de un compañero y en el servidor.

Vocabulario mínimo:

Palabra	Qué es
Imagen	La «receta» ya cocinada, de solo lectura. Se construye con un <code>Dockerfile</code>
Contenedor	Una imagen en ejecución. Puedes tener varios de la misma imagen
Volumen	Carpeta persistente: lo que el contenedor escribe ahí sobrevive cuando lo borras
Puerto publicado	<code>-p 4000:4000</code> : lo que escucha dentro, visible desde fuera
Compose	<code>compose.yaml</code> : varios contenedores descritos en un archivo; <code>docker compose up</code> los levanta todos
Registro	Donde viven las imágenes (Docker Hub, GHCR). <code>docker pull</code> las baja

En este curso Docker sirve para dos cosas: levantar el **gateway de IA** (`jmd`) con un solo comando, y más adelante ejecutar código de los alumnos en contenedores aislados.

Podman es una alternativa compatible (mismos comandos, sin demonio como root). Todo lo del curso funciona igual con `podman` y `podman compose`.

Instalar Docker

Sistema	Cómo
Windows 10/11	Docker Desktop . Requiere WSL 2: si no lo tienes, <code>wsl --install</code> en PowerShell como administrador, reinicia, y luego instala Docker Desktop con «Use WSL 2 based engine». En <i>Settings</i> → <i>Resources</i> → <i>WSL integration</i> , activa tu Ubuntu
macOS	Docker Desktop (elige Apple Silicon o Intel). Alternativas más ligeras: OrbStack o Colima
Linux	Docker Engine siguiendo docs.docker.com/engine/install. Después: <code>sudo usermod -aG docker \$USER</code> y vuelve a entrar en la sesión para no usar <code>sudo</code>

Comprobación:

```
docker --version
docker run --rm hello-world
```

Si `hello-world` imprime «Hello from Docker!», el demonio está arriba y puedes bajar imágenes.

Prueba real (opcional): levanta el gateway tú mismo.

```
git clone https://github.com/davrv93/jmd && cd jmd
cp .env.example .env          # pon ADMIN_TOKEN y las claves que tengas
docker compose up -d --build  # o: podman compose up -d --build
# UI en http://localhost:4000/ui/
```

3. jmd: instalar y conectar

Qué es jmd

`jmd` es nuestro **CLI** (programa de terminal), escrito en Rust, para hablar con el **gateway de IA** de la clase. El gateway es un servidor compatible con la API de OpenAI y con la de Anthropic que **elige el modelo por ti**, hace *fallback* cuando un proveedor gratuito devuelve 429, lleva las cuotas y la telemetría, y tiene una UI de gestión. Tu agente (Claude Code u OpenCode) se conecta al gateway y no a un proveedor concreto: por eso todos usamos la misma cuenta y las mismas cuotas.

Instalar

No hace falta Rust: el instalador baja el binario de las *releases*, comprueba su SHA-256 y lo deja en tu PATH sin pedir permisos de administrador.

Windows (PowerShell), en la terminal de VS Code:

```
irm https://raw.githubusercontent.com/davrv93/jmd/main/install.ps1 | iex
```

Queda en `%LOCALAPPDATA%\Programs\jmd\jmd.exe`. **Cierra y abre la terminal** para que la tome.

WSL (Ubuntu), macOS y Linux:

```
curl -fsSL https://raw.githubusercontent.com/davrv93/jmd/main/install.sh | sh
```

Queda en `~/.local/bin/jmd`. Si no está en el PATH, el instalador te dice la línea que debes añadir a `~/.bashrc` o `~/.zshrc`.

WSL y Windows son dos sistemas distintos. Instala `jmd` y el agente **donde corre tu terminal**.

Si VS Code pone «WSL: Ubuntu» abajo a la izquierda, instala la versión Linux dentro de WSL.

Detalles y problemas de red entre WSL y Windows: [docs/INSTALACION.md](#).

Conectar

```
jmd login          # URL del gateway y clave (las da el instructor)
jmd status         # gateway, tokens, Claude Code, OpenCode, RTK y caveman
jmd setup claude   # Claude Code → gateway (y ofrece el hook de RTK)
jmd setup opencode # OpenCode → gateway
```

Comandos que usarás a diario:

Comando	Para qué
<code>jmd</code>	Modo interactivo: chat con streaming y los comandos de abajo
<code>jmd models</code> · <code>jmd providers</code> · <code>jmd quotas</code>	Qué modelos hay, qué proveedores, cuánta cuota queda
<code>jmd route "..."</code>	Sin llamar a nadie: a qué agente y a qué cadena de modelos iría tu petición
<code>jmd chat "..."</code>	Una pregunta
<code>jmd stats</code>	Uso, éxito, latencia y ahorro de tokens por estilo y cliente
<code>jmd style off lite full ultra</code>	Respuestas más cortas para todos los clientes

Todos aceptan `--json`.

Instalar el agente

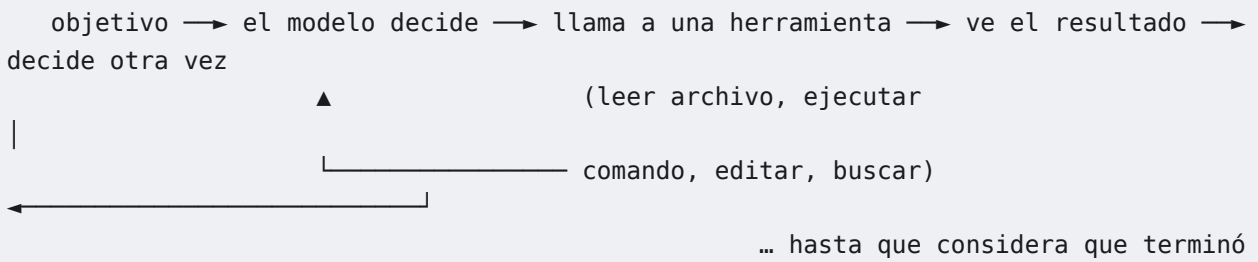
- **Claude Code:** `npm i -g @anthropic-ai/claude-code` (necesitas Node.js 18+; en Windows nativo, también Git for Windows). Después `jmd setup claude` y arranca con `claude`.
- **OpenCode:** sigue <https://opencode.ai>. Después `jmd setup opencode` y arranca con `opencode`.

Comprobación final: `jmd status` con todo en verde.

4. Conceptos: agente, proveedor, modelo, parámetros

Qué es un agente de IA

Un **modelo de lenguaje** solo hace una cosa: recibe texto y devuelve texto. Un **agente** es un programa que pone ese modelo en un **bucle con herramientas**:



Claude Code y OpenCode son agentes **de programación**: sus herramientas son leer y escribir archivos, ejecutar comandos en tu terminal, buscar en el repo, y las que tú le añadas (MCP). Lo que distingue a un agente de un chat es que **actúa** sobre tu proyecto y comprueba el resultado de cada acción. Por eso tu trabajo cambia: pasas de escribir cada línea a **definir el objetivo, dar contexto, revisar lo que hace y decidir**. A eso le llamamos *vibecoding* cuando se hace bien; cuando se hace mal, es aceptar sin leer.

Reglas que sostendremos todo el curso:

1. **Lee el diff antes de aceptar**. El agente propone; tú decides.
2. **Contexto claro y corto**: qué quieres, dónde está el código, cómo se prueba.
3. **Trabaja en git**: cada paso en un commit; si el agente rompe algo, vuelves atrás.
4. **Dale una forma de comprobar** (una prueba, un comando): un agente que puede verificar se equivoca menos.

Qué es un proveedor (provider)

Un **proveedor** es quien aloja el modelo y expone una **API** por la que lo llamas: OpenAI, Anthropic, Google (AI Studio / Gemini), OpenRouter, OpenCode Zen, DeepSeek, Groq... o tu propia máquina con **Ollama**. Cada proveedor tiene su URL base, su clave (`API key`), sus límites de uso (cuota, peticiones por minuto) y su precio. Casi todos hablan el **formato de OpenAI** (`POST /v1/chat/completions`), que se ha convertido en el estándar de hecho; Anthropic tiene el suyo (`/v1/messages`), que es el que usa Claude Code.

En el gateway, un proveedor son tres líneas:

```
providers:
  openrouter:
    base_url: https://openrouter.ai/api/v1
    api_key_env: OPENROUTER_API_KEY
```

OpenRouter merece mención: es un *agregador*, un proveedor que da acceso a cientos de modelos de muchos laboratorios con una sola clave, con una capa gratuita (`:free`) que usaremos mucho.

Qué es un modelo

Un **modelo** es la red neuronal entrenada que produce el texto: `gemini-2.5-flash`, `claude-sonnet`, `nvidia/nemotron-3-super` ... Lo que importa de un modelo para elegirlo:

Rasgo	Qué significa
Capacidades	texto, código, razonamiento, <i>tools</i> (puede pedir llamar a una función), JSON, imagen, audio, vídeo, PDF
Ventana de contexto	Cuántos tokens caben entre lo que le mandas y lo que responde (p. ej. 128 k o 1 M)
Costo	Por millón de tokens de entrada y de salida; los <code>: free</code> cuestan 0 pero tienen cuota
Velocidad	Tokens por segundo y latencia hasta el primer token
Calidad	En qué tareas acierta. Se mide con <i>benchmarks</i> , pero sobre todo con tu propia telemetría

Un **token** es la unidad en la que el modelo lee y escribe: trozos de palabra, más o menos ¾ de palabra en inglés y algo menos en español. Todo se cobra y se limita en tokens.

El mismo modelo puede estar en varios proveedores: en el gateway, `nemotron-super` tiene dos **deployments** (OpenRouter y OpenCode Zen) y si uno falla se prueba el otro. Y un **agente del gateway** (`coding`, `reasoning`, `cheap` ...) es una **cadena de modelos** ordenada: si el primero devuelve 429, va al siguiente. Cuando pides `model: auto`, el *router* decide la cadena por ti.

```
jmd models           # los modelos y su estado
jmd route "refactoriza este módulo en Rust"  # a qué cadena iría, sin gastar nada
```

Qué son los parámetros

Los **parámetros de generación** van en cada petición y cambian cómo responde el mismo modelo:

Parámetro	Qué hace	Cuándo tocarlo
<code>temperature</code>	Aleatoriedad al elegir cada token. 0 = casi determinista; 1 = creativo	Código: bajo (0.1–0.3). Ideas: alto
<code>top_p</code>	Muestreo por núcleo: solo los tokens que suman esa probabilidad	Alternativa a <code>temperature</code> ; no subas los dos
<code>max_tokens</code>	Tope de tokens de salida	Evita respuestas interminables y cuida la cuota
<code>stop</code>	Secuencias que cortan la respuesta	Salidas con formato fijo
<code>system</code> / mensaje de sistema	Instrucciones de rol y estilo que valen para toda la conversación	Siempre: es donde va «responde corto», «usa español»...
<code>tools</code>	Funciones que el modelo puede pedir llamar (con su esquema JSON)	Lo que hace agente a un chat
<code>response_format</code>	Forzar JSON (o un esquema)	Cuando otro programa va a leer la respuesta
<code>seed</code>	Reproducibilidad (si el proveedor lo soporta)	Pruebas

En el gateway, cada agente lleva sus parámetros por defecto; por ejemplo:

```
agents:
  coding:
    chain: [laguna, muse-spark, nemotron-super, inkling, gemini-flash]
    priority: quality
    params: {temperature: 0.2}
```

No confundas estos **parámetros de generación** con los **parámetros del modelo** (los miles de millones de «pesos» que se ajustaron al entrenarlo; «un modelo de 70 B» tiene 70 000 millones). Cuando alguien dice «el modelo tiene más parámetros», habla de tamaño; cuando dice «baja la temperatura», habla de la petición.

5. Ahorrar tokens: RTK y caveman

Un agente gasta tokens en dos direcciones: lo que **le entra** (tu prompt, los archivos que lee, la salida de cada comando) y lo que **sale** (sus respuestas). Con cuotas gratuitas, eso es lo que marca cuánto puedes trabajar al día.

Qué es RTK

RTK (*Rust Token Killer*, `rtk-ai/rtk`) actúa sobre lo que **entra**: se engancha a Claude Code con un *hook* y **comprime la salida de los comandos** antes de que llegue al modelo. Quita códigos de color, líneas repetidas, resúmenes de `git status` o `cargo build` que el modelo no necesita, y recorta por el medio conservando el final, donde suele estar el error. El modelo ve lo mismo que necesitaba ver, con muchos menos tokens.

```
brew install rtk                # macOS
cargo install --git https://github.com/rtk-ai/rtk # cualquier sistema con Rust
jmd setup claude                # ofrece instalar el hook (rtk init
--global)
rtk gain                        # cuánto ha ahorrado
```

El gateway tiene la misma idea del lado del servidor (`savings.compress_tool_output`), para clientes que no tienen hooks.

Qué es caveman

caveman (`JuliusBrussee/caveman`) actúa sobre lo que **sale**: es un *plugin* de Claude Code que instruye al modelo para responder **como un cavernícola**: frases cortas, sin cortesías ni repeticiones, solo lo esencial. Ahorra tokens de salida y, de paso, respuestas más rápidas.

El gateway ofrece lo mismo para **cualquier** cliente con `jmd style lite|full|ultra` (`full` equivale a *caveman*). Si tienes el plugin y el estilo del gateway, el recorte se aplica dos veces: deja uno de los dos. `jmd status` te avisa.

```
jmd style full    # para todos los clientes (Claude Code, OpenCode, jmd chat)
jmd stats         # compara tokens de salida por estilo y por cliente
```

6. Qué es MCP

MCP (*Model Context Protocol*) es un **protocolo abierto** para conectar un agente con **herramientas y datos externos** de forma estándar. Antes, cada agente integraba cada servicio a su manera; con MCP, un **servidor MCP** expone sus capacidades una vez y cualquier **cliente MCP** (Claude Code, OpenCode, el escritorio de Claude, Cursor...) las usa.

Un servidor MCP expone tres cosas:

	Qué es	Ejemplo
Tools	Funciones que el agente puede llamar, con esquema JSON	<code>list_courses</code> , <code>submit_assignment</code> , <code>query_database</code>
Resources	Datos que el agente puede leer, identificados por URI	<code>lms://lesson/pa-01</code> , un archivo, una fila
Prompts	Plantillas de instrucciones reutilizables	«revisa este PR con nuestra guía»

Se habla por **JSON-RPC**, y el transporte es **stdio** (el cliente arranca el servidor como proceso local) o **HTTP** (*streamable HTTP*, para servidores remotos con autenticación).

En Claude Code, un servidor se registra así:

```
claude mcp add --transport http lms https://lms.tu-dominio/mcp
claude mcp add github -- npx -y @modelcontextprotocol/server-github
```

Y en OpenCode, en `opencode.json`:

```
{ "mcp": { "lms": { "type": "remote", "url": "https://lms.tu-dominio/mcp" } } }
```

En este curso, el propio LMS tendrá un servidor MCP: tu agente podrá listar tus cursos, leer los materiales de una sesión, entregar una tarea y publicar una pregunta sin salir de la terminal (`jmd setup claude` lo registrará solo).

7. Qué es una skill

Una **skill** es una **carpeta con un `SKILL.md`** que enseña al agente a hacer algo concreto: cómo trabaja tu equipo, cómo se despliega este proyecto, cómo se escribe un informe en tu formato. El archivo lleva un front matter YAML con `name` y `description` y, debajo, instrucciones en Markdown; puede llevar también scripts y archivos de referencia.

Lo importante es **cómo se carga**: el agente solo lee al inicio el nombre y la descripción de cada skill (unas líneas). Cuando tu pedido encaja con una descripción, carga el `SKILL.md` completo, y solo entonces los archivos que este referencia. Es **contexto bajo demanda**: puedes tener cien skills instaladas sin gastar tokens hasta que una hace falta.

Dónde viven:

Alcance	Claude Code
Un proyecto	<code>.claude/skills/<nombre>/SKILL.md</code> (se versiona con el repo)
Tu usuario	<code>~/.claude/skills/<nombre>/SKILL.md</code>
Plugins	Un plugin puede traer varias skills

Se invocan solas cuando el pedido encaja, o a mano con `/nombre`.

Ejemplos de skills

1. Saludo (la de la tarea):

```
---
name: saludo
description: Saluda al alumno por su nombre y le dice qué clase toca. Úsala cuando pidan «salúdame» o «qué toca hoy».
---
Pregunta el nombre si no lo sabes. Responde en español, en dos líneas:
1. Un saludo con el nombre.
2. La clase de hoy, leída de `clases.md` si existe; si no, di que no hay archivo.
```

2. Commits convencionales:

```
---
name: commit
description: Escribe mensajes de commit con Conventional Commits (feat, fix, docs...) en español y en imperativo. Úsala al pedir «haz commit».
---
1. Corre `git diff --staged`. Si no hay nada preparado, pregunta qué incluir.
2. Tipo: feat · fix · docs · refactor · test · chore. Ámbito entre paréntesis si es claro.
3. Primera línea ≤ 72 caracteres, en imperativo («añade», no «añadido»).
4. Cuerpo: el porqué, no el qué. Enséñame el mensaje antes de ejecutar `git commit`.
```

3. Revisión de PR con la guía del equipo (con archivo de referencia):

```
---
name: revisar-pr
description: Revisa un pull request con la guía de estilo del equipo en GUIA.md. Úsala cuando pidan revisar un PR o una rama.
---
Lee `GUIA.md` de esta carpeta. Para cada archivo del diff, comprueba la guía y señala solo problemas reales con `archivo:línea`. Termina con un veredicto: aprobar / cambios necesarios.
```

4. Documentos del curso (skill con script):

```
---
name: informe
description: Genera el informe semanal del alumno en Markdown con la plantilla del
curso. Úsala cuando pidan «informe de la semana».
---
Ejecuta `python scripts/plantilla.py --semana N` para obtener la plantilla y
rellénala con lo hecho (commits de la semana: `git log --since="1 week ago"`).
```

Fíjate en el patrón: la **descripción dice cuándo usarla** (eso es lo que el agente lee siempre), y el cuerpo dice **cómo**, en pasos cortos y verificables. Los ejemplos de Anthropic (hojas de cálculo, PDF, documentos, diseño de artefactos) están en el repositorio de materiales.

Tareas

Tarea 1 · Tu entorno listo y tu primera skill (fecha límite: ver «Tareas» del curso). El enunciado completo y la rúbrica están en la tarea. Resumen:

1. Instala VS Code, Docker y `jmd` ; conecta un agente.
2. Crea un repositorio `clase-01` con `EVIDENCIA.md` (salida de `code --version` , `docker run --rm hello-world` , `jmd status` y una captura del agente) y `.claude/skills/saludo/SKILL.md` con tu skill.
3. Entrega el repo (URL + commit) desde el LMS o con `jmd submit tarea-01`.

Para la próxima clase: ten el entorno listo. Empezaremos a construir un proyecto con el agente desde cero, en git, y veremos cómo darle contexto (CLAUDE.md / AGENTS.md).

Tarea 1 · Tu entorno listo y tu primera skill

Fecha límite: 16 de octubre de 2026, 4:59 a. m. · **Puntaje:** 20 puntos · entrega en el LMS o con `jmd submit tarea-01`

Qué entregar

Un repositorio git (GitHub, GitLab o Forgejo, público o con acceso para el instructor) llamado `clase-01`, con:

1. `EVIDENCIA.md` con la salida literal de estos comandos (bloques de código) y una captura de pantalla del agente corriendo en la terminal de VS Code:

```
code --version
docker run --rm hello-world
jmd status
```

2. `.claude/skills/saludo/SKILL.md`: tu primera skill. Debe tener `name` y `description` en el front matter y, debajo, instrucciones en pasos. Pídele al agente «salúdame» y pega su respuesta en `EVIDENCIA.md`.
3. `REFLEXION.md` (10–15 líneas): qué le pediste al agente en el paso «practicar», qué archivos creó, qué comandos quiso ejecutar, qué aceptaste y qué no, y por qué.

Cómo entregar

- Desde el LMS: en esta tarea, pega la URL del repositorio y el SHA del commit.
- Desde la terminal, dentro del repo: `jmd submit tarea-01` (envía el `origin` y el `HEAD`).

Se califica el commit exacto que entregues; si subes cambios después, vuelve a entregar.

Rúbrica

Entorno instalado con evidencia (VS Code, Docker, jmd, agente)	Sin evidencia: 0 · Parcial (falta alguna pieza o comprobación): 4 · Completo (las cuatro comprobaciones en verde): 8
Skill `saludo` correcta (front matter válido, descripción que dice cuándo usarla, instrucciones claras)	No entregada: 0 · Funciona pero la descripción o los pasos son vagos: 4 · Funciona y está bien escrita: 8
Reflexión (qué hizo el agente, qué aceptaste y qué no, y por qué)	No hay: 0 · Superficial: 2 · Concreta: 4